

RUTA PARALELA DE IA

para avanzar junto al Common Core de 42 Barcelona

Plan paso a paso: fundamentos -> ML -> Deep Learning -> LLMs -> RAG -> agentes -> ingeniería aplicada

Diseñada para estudiar en paralelo a 42, no para sustituir 42.

Punto de partida: post-Milestone 2. Prioridad actual: seguir avanzando en el Common Core y preparar Milestone 3.

Versión 1.0 - 24 de agosto de 2026

1. Cómo usar esta ruta

Este documento no es una lista de tecnologías para aprender. Es un orden de aprendizaje. La regla principal es sencilla: **no estudies un bloque avanzado porque suene interesante si todavía no cumples el checkpoint del bloque anterior.**

Objetivo profesional: Convertirte primero en un software engineer con fundamentos sólidos y después especializarte en IA aplicada. No perseguir “saber de IA” de forma superficial.

Tiempo semanal recomendado

- Semana normal: 3-5 horas fuera de 42.
- Semana pesada de proyecto/examen: 0-2 horas. Pausar no es retroceder.
- Máximo razonable: 6 horas. No sacrifiques progreso real de 42 para acumular vídeos.
- Reparto aproximado: 30% teoría / 70% práctica, preguntas, ejercicios y explicación con tus palabras.

Regla de dominio

Un tema se considera “suficientemente aprendido” cuando puedes cumplir las cuatro condiciones:

- Explicarlo sin mirar apuntes en lenguaje sencillo.
- Resolver un ejercicio pequeño sin copiar una solución.
- Predecir al menos un error típico o caso límite.
- Relacionarlo con el siguiente concepto de la ruta.

Tu ciclo de estudio con ChatGPT

1. Dime el identificador del tema: por ejemplo, “Ruta IA - B1.3, producto escalar. Vamos con esto”.
2. Yo empiezo comprobando qué entiendes con 2-4 preguntas, no soltando una clase completa.
3. Corregimos solo los huecos importantes y construimos la intuición.
4. Haces un mini-ejercicio o predices un resultado.
5. Revisamos tu razonamiento y tus errores.
6. Hacemos un checkpoint final. Si lo superas, pasas al siguiente punto.

Frase suficiente para retomar: No necesitas copiar un prompt largo. Con “vamos con B2.4 de mi ruta de IA” podemos trabajar ese punto paso a paso.

2. Mapa general

La ruta está dividida en bloques. Los primeros se estudian ya; los bloques de LLM, RAG y agentes se activan cuando estés cerca de esos proyectos en 42.

Bloque	Qué aprendes	Cuándo	Duración orientativa	Checkpoint
B0	Método y entorno	Ahora	1 sesión	Sabes cómo estudiar y medir progreso
B1	NumPy + intuición matemática	Ahora	4-6 semanas	Vectores, matrices, dot, media/varianza, gradiente
B2	Machine Learning clásico	Después de B1	6-10 semanas	Entrenas y evalúas modelos sin tratar fit() como magia
B3	Deep Learning + PyTorch	Después de B2	6-8 semanas	Entiendes forward/loss/backprop/optimizer
B4	Fundamentos de LLM	1-2 proyectos antes de Call Me Maybe	3-5 semanas	Tokens -> logits -> sampling; transformer conceptual
B5	Retrieval y RAG	Antes de Against the Machine	4-6 semanas	TF-IDF/BM25/ embeddings/top-k/ Recall@k
B6	Agentes y tools	Antes de Agent Smith	4-6 semanas	Loop act/observe, tools, estado, sandbox y evaluación
B7	AI engineering	Continuo	Durante 42 y portfolio	API, tests, DB, Docker, logging, deploy

La relación con 42

```
42:      C / Python / algoritmos / threads / proyectos / Docker / web
          |
Ruta IA:  matemáticas -> ML -> PyTorch -> LLM -> RAG -> agentes
          |
          +-----> perfil de AI / ML Engineer aplicado
```

42 te da el entorno de ingeniería y proyectos. Esta ruta rellena principalmente la teoría de ML/IA y te prepara justo antes de los proyectos que la necesitan.

3. Las primeras 8 semanas: exactamente por dónde empezar

Importante: Las “semanas” son orientativas. Si un tema te requiere 10 días, usa 10 días. No avances por calendario.

Semana 1 - NumPy: arrays y forma de los datos

Temas

- B1.1 Qué problema resuelve NumPy y diferencia entre list y ndarray.

- B1.2 Crear arrays 1D y 2D; dtype, ndim, shape, size.
- B1.3 Indexing y slicing.
- B1.4 Ejes: qué significa axis=0 y axis=1.

Práctica mínima

- Predecir shape antes de ejecutar.
- Crear vector y matriz a partir de listas.
- Acceder a una fila, una columna y una submatriz.

Checkpoint: Puedes mirar un array y decir dimensiones, shape y qué representa cada eje.

Semana 2 - Operaciones vectoriales

Temas

- B1.5 Suma/resta de vectores y multiplicación por escalar.
- B1.6 Producto escalar (dot product).
- B1.7 Norma/magnitud de un vector.
- B1.8 Media, suma y desviación estándar con arrays.
- B1.9 Intuición de cosine similarity.

Práctica mínima

- Calcular un dot product a mano y luego comprobarlo con NumPy.
- Explicar por qué dos vectores deben tener dimensiones compatibles.
- Comparar dos vectores con producto escalar y coseno.

Checkpoint: Puedes explicar qué mide un producto escalar y por qué aparecerá en embeddings/retrieval.

Semana 3 - Funciones, derivada y gradiente

Temas

- B1.10 Función $f(x)$: entrada, salida y parámetros.
- B1.11 Pendiente y derivada como cambio local.
- B1.12 Derivadas parciales.
- B1.13 Gradiente como vector de derivadas.
- B1.14 Gradient descent: “mover parámetros para reducir error”.

Práctica mínima

- Dibujar mentalmente una parábola y decidir hacia dónde moverte para bajar.
- Calcular pendientes sencillas en puntos concretos.
- Explicar sin fórmula qué representa un gradiente.

Checkpoint: Entiendes por qué una loss puede guiar la actualización de parámetros.

Semana 4 - Estadística y probabilidad mínimas

Temas

- B1.15 Media, mediana, varianza y desviación estándar.
- B1.16 Población vs muestra.
- B1.17 Probabilidad básica y eventos.
- B1.18 Distribución: intuición, no memorización de fórmulas.
- B1.19 Correlación no implica causalidad.

Práctica mínima

- Calcular media/varianza de un conjunto pequeño.

- Comparar dos distribuciones con la misma media y distinta dispersión.
- Interpretar una probabilidad condicional sencilla.

Checkpoint: Puedes describir datos con media y dispersión y entiendes qué significa una muestra.

Semana 5 - Modelo mental de Machine Learning

Temas

- B2.1 Qué es ML y qué NO es.
- B2.2 Features X y target y.
- B2.3 Supervised vs unsupervised.
- B2.4 Regression vs classification.
- B2.5 Train / validation / test.
- B2.6 Baseline y data leakage.

Práctica mínima

- Clasificar 10 problemas reales como regresión/clasificación/clustering.
- Dado un dataset, identificar X e y.
- Explicar por qué no debes evaluar con los mismos datos de entrenamiento.

Checkpoint: Sabes plantear correctamente un problema antes de elegir modelo.

Semana 6 - Error, aprendizaje y evaluación

Temas

- B2.7 Prediction, error y loss.
- B2.8 Linear regression: intuición.
- B2.9 Logistic regression: intuición.
- B2.10 Confusion matrix.
- B2.11 Accuracy, precision, recall y F1.
- B2.12 Overfitting y underfitting.

Práctica mínima

- Resolver una confusion matrix pequeña a mano.
- Elegir precision o recall según un caso práctico.
- Detectar un ejemplo de overfitting.

Checkpoint: Puedes justificar una métrica y explicar por qué un modelo puede memorizar.

Semana 7 - Árboles, generalización y validación

Temas

- B2.13 Decision trees.
- B2.14 Random forest: intuición de ensemble.
- B2.15 Cross-validation.
- B2.16 Regularization: idea general.
- B2.17 Hiperparámetros vs parámetros.

Práctica mínima

- Comparar un árbol muy profundo con uno pequeño.
- Explicar por qué combinar árboles puede mejorar estabilidad.
- Explicar qué intenta medir cross-validation.

Checkpoint: Ya no juzgas un modelo solo por “accuracy alta”.

Semana 8 - Mini-proyecto ML

Temas

- B2.18 Elegir un dataset pequeño.
- B2.19 Separar train/test.
- B2.20 Entrenar baseline + 2 modelos.
- B2.21 Elegir métricas.
- B2.22 Analizar errores.
- B2.23 README corto: problema, datos, decisión y resultado.

Práctica mínima

- No busques un proyecto espectacular.
- El objetivo es poder defender cada decisión.
- Usa scikit-learn después de entender el pipeline.

Checkpoint: Puedes explicar el proyecto entero sin mirar código y señalar qué mejorarías.

4. Cómo estudiar: qué recurso usar para cada cosa

No necesitas elegir entre YouTube, documentación o ChatGPT. Cada herramienta sirve para una parte diferente. El error es usar una sola para todo.

Necesidad	Mejor herramienta	Cómo usarla	Qué evitar
Intuición visual	3Blue1Brown	Ver un vídeo, pausar y dibujar/explicar lo entendido.	Maratón de vídeos.
Estadística/ML explicado claro	StatQuest	Un vídeo por concepto + ejercicio propio.	Creer que entender el vídeo = saber aplicarlo.
Curso estructurado	DeepLearning.AI / Andrew Ng	Seguir módulos concretos y hacer labs.	Coleccionar certificados.
API y uso correcto	Documentación oficial	Buscar una función cuando ya sabes qué concepto necesitas.	Leer documentación como novela.
Práctica	Python/NumPy/scikit/PyTorch	Escribir, predecir resultados, romper cosas, depurar.	Copiar notebooks completos.
Mentoría activa	ChatGPT	Diagnóstico -> explicación -> ejercicio -> revisión -> checkpoint.	Pedir “explícame todo ML” y leer pasivamente.

Protocolo de una sesión de 90 minutos

1. 10 min - Recuperación: sin apuntes, escribe lo que recuerdas del tema anterior.
2. 25 min - Nueva idea: vídeo/lectura breve, con pausas.
3. 35 min - Práctica: ejercicios, cálculos o código pequeño.
4. 15 min - Explicación: cuéntamelo con tus palabras y responde preguntas.
5. 5 min - Registro: anota qué no entiendes todavía y el siguiente punto.

Reglas contra el aprendizaje pasivo

- Después de 20-30 minutos de contenido, debe existir alguna acción: escribir, calcular, programar o explicar.
- Si ves dos explicaciones distintas y sigues perdido, deja de buscar vídeos: tráeme tu modelo mental y lo depuramos.

- No copies código que no puedas explicar línea por línea.
- No uses IA para “terminar” un mini-proyecto. Úsala para detectar el hueco que te impide terminarlo.
- Mantén un archivo notes.md con conceptos, errores y preguntas; no con transcripciones de cursos.

5. B2 - Machine Learning clásico

Objetivo: entender el pipeline completo de ML antes de pasar a redes neuronales. Aquí puedes usar scikit-learn, pero siempre sabiendo qué problema resuelve cada llamada.

B2.A - Problema y datos

- features/target
- regresión/clasificación/clustering
- train/validation/test
- preprocessing básico
- data leakage

B2.B - Modelos esenciales

- linear regression
- logistic regression
- k-nearest neighbors (intuición)
- decision trees
- random forests
- k-means

B2.C - Evaluación

- MSE/MAE
- confusion matrix
- accuracy
- precision
- recall
- F1
- ROC-AUC a nivel conceptual
- cross-validation

B2.D - Generalización

- overfitting/underfitting
- bias/variance
- regularization
- hiperparámetros
- pipelines

B2.E - Proyecto de cierre

- un dataset pequeño
- baseline
- 2-3 modelos
- comparación de métricas

- error analysis
- README defendible

No avances a B3 si...: sigues pensando “el modelo ha dado 93%” sin poder explicar qué datos vio, qué métrica es, contra qué baseline comparas y cómo sabes que generaliza.

6. B3 - Deep Learning y PyTorch

Objetivo: que una red neuronal deje de ser una caja negra. No necesitas entrenar modelos enormes; necesitas comprender el ciclo de entrenamiento.

B3.1 - Tensors

- tensor vs ndarray
- shape/dtype/device
- operaciones y broadcasting

B3.2 - Red neuronal

- neuronas/capas
- weights y bias
- activaciones
- forward pass

B3.3 - Aprendizaje

- loss function
- gradient descent
- backpropagation conceptual
- autograd

B3.4 - Entrenamiento

- Dataset/DataLoader
- batch
- optimizer
- learning rate
- train/eval mode

B3.5 - Diagnóstico

- train loss vs validation loss
- overfitting
- normalización básica
- guardar/cargar modelo

B3.6 - Mini-proyecto

- clasificador pequeño
- loop de entrenamiento escrito por ti
- métricas

- análisis de errores

Checkpoint B3: Dado un training loop, puedes señalar dónde ocurre forward, loss, backward, actualización de parámetros y evaluación.

7. B4 - Fundamentos de LLM (antes de Call Me Maybe)

Activa este bloque cuando Call Me Maybe esté aproximadamente 1-2 proyectos por delante. El objetivo es llegar al subject entendiendo el vocabulario básico.

B4.1 - Del texto al modelo

- token
- tokenizer
- token ID
- vocabulario
- secuencia/contexto

B4.2 - De la salida al siguiente token

- logits
- softmax
- probabilidad
- sampling
- temperature
- top-k/top-p a nivel conceptual

B4.3 - Representaciones

- embedding
- dimensión
- similaridad
- por qué palabras/conceptos pueden representarse como vectores

B4.4 - Transformer conceptual

- decoder-only
- attention
- query/key/value a nivel intuitivo
- residual/MLP solo a nivel de mapa

B4.5 - Generación controlada

- structured output
- JSON schema
- constrained decoding
- máscaras/restricción de tokens

texto -> tokenizer -> token IDs -> modelo -> logits -> softmax -> elección de token -> repetir

Checkpoint B4: Puedes explicar la cadena anterior y qué significa “modificar/restringir logits” sin decir únicamente que son “probabilidades”.

8. B5 - Retrieval y RAG (antes de Against the Machine)

Aquí la prioridad es entender retrieval antes de añadir un LLM. Un RAG malo suele empezar por una recuperación mala.

B5.1 - Information Retrieval

- corpus
- document
- chunk
- query
- index
- ranking
- top-k

B5.2 - Búsqueda léxica

- bag of words
- term frequency
- inverse document frequency
- TF-IDF
- BM25: intuición y por qué mejora ranking

B5.3 - Búsqueda vectorial

- embeddings
- cosine similarity
- dense retrieval
- vector index a nivel conceptual

B5.4 - Evaluación

- relevant/not relevant
- precision@k
- recall@k
- MRR a nivel opcional
- gold set

B5.5 - RAG

- retrieve -> assemble context -> generate
- chunking
- context window
- citations/evidence
- hallucination vs evidence

B5.6 - Mini-proyecto previo

- buscador sobre 10-50 archivos
- primero TF-IDF/BM25
- luego embeddings si procede
- sin LLM al principio

Checkpoint B5: Ante una respuesta RAG mala, sabes separar: ¿falló retrieval, ranking, chunking, contexto o generación?

9. B6 - Agentes y herramientas (antes de Agent Smith)

No estudies “agentes” como marketing. Trátalos como un sistema de software que mantiene estado, elige acciones, ejecuta herramientas, observa resultados y decide qué hacer después.

B6.1 - Tool calling

- tool/function schema
- argumentos estructurados
- validación
- resultado de herramienta

B6.2 - Loop del agente

- state
- action
- observation
- stop condition
- errores y retries

B6.3 - Context engineering

- qué información entra
- qué se descarta
- historial
- resúmenes
- límites de contexto

B6.4 - Ejecución segura

- subprocess
- timeouts
- filesystem limitado
- network policies
- sandboxing conceptual

B6.5 - Protocolos

- MCP a nivel conceptual
- cliente/servidor
- descubrimiento de herramientas/recursos

B6.6 - Evaluación

- task success
- reproducibilidad
- traces/logs
- failure modes

```
state -> choose action -> execute tool -> observe -> update state -> repeat -> stop
```

Checkpoint B6: Puedes diseñar un agente pequeño sin framework y señalar claramente estado, herramientas, condición de parada y riesgos.

10. B7 - Ingeniería aplicada: lo que convierte IA en un producto

Este bloque no es un curso separado. Se va incorporando conforme 42 te exponga a Docker, web y proyectos de equipo.

Backend

- HTTP/REST
- FastAPI o equivalente
- validación de inputs
- async solo cuando entiendas por qué

Datos

- SQL
- PostgreSQL
- esquema
- índices básicos
- migraciones a nivel práctico

Calidad

- tests unitarios/integración
- logging
- configuración
- manejo de errores
- type hints

Infra

- Docker
- variables de entorno
- secrets
- CI/CD básico
- observabilidad

IA en producción

- latencia

- coste
- caché
- rate limits
- evaluaciones
- fallbacks

11. Proyectos paralelos: pocos y con intención

No necesitas crear un proyecto nuevo para cada concepto. Tu portfolio debe crecer cuando ya tengas base para construir algo defendible.

Proyecto	Momento	Qué demuestra
P1 - ML pequeño	Después de B2	Dataset tabular; baseline + 2 modelos; métricas; análisis de errores.
P2 - Deep Learning pequeño	Después de B3	Clasificación sencilla con PyTorch; training loop propio; curvas y error analysis.
P3 - Buscador/Retrieval	Antes o durante B5	Indexar documentos/código y recuperar top-k; comparar TF-IDF/BM25/embeddings.
P4 - AI service	Después de B5/B6	API con LLM/RAG/tools, tests, DB si aporta valor, Docker y evaluación.

Cada proyecto debe poder defender estas preguntas

- ¿Qué problema intentabas resolver y por qué esta solución?
- ¿Qué alternativa consideraste?
- ¿Qué métrica utilizaste y por qué?
- ¿Cuál fue el principal fallo?
- ¿Qué harías distinto con una semana más?
- ¿Qué parte entiendes a nivel interno y qué parte delegas a una librería?

12. Recursos recomendados (pocos y ordenados)

Regla: para cada fase elige un recurso principal. Los demás son apoyo cuando una explicación no te encaja.

Ahora: B1 - NumPy y matemáticas

- **NumPy - Absolute Basics for Beginners** - Documentación oficial para arrays, shape, indexing y operaciones. [Abrir recurso](#)
- **3Blue1Brown - Linear Algebra** - Tu recurso visual para vectores, matrices y transformaciones. No hace falta ver toda la serie de golpe. [Abrir recurso](#)
- **StatQuest - Video Index** - Estadística y ML organizados aproximadamente de básico a avanzado. [Abrir recurso](#)
- **DeepLearning.AI - Mathematics for ML & Data Science** - Opcional como curso estructurado. No necesitas completarlo entero antes de ML; úsalo por módulos si te falta base. [Abrir recurso](#)

B2 - Machine Learning

- **DeepLearning.AI - Machine Learning Specialization** - Curso estructurado de Andrew Ng. Úsalo como columna vertebral si quieres una secuencia guiada. [Abrir recurso](#)
- **scikit-learn - User Guide** - Referencia de implementación, modelos, métricas y evaluación. Consultar por necesidad, no memorizar. [Abrir recurso](#)

B3 - Deep Learning

- **3Blue1Brown - Neural Networks** - Intuición visual de redes, gradient descent, backprop y transformers. [Abrir recurso](#)
- **PyTorch - Learn the Basics** - Ruta oficial: tensors, datasets, modelos, autograd, optimización y guardado. [Abrir recurso](#)

B4 en adelante - LLMs

- **Hugging Face - LLM Course (ES)** - Úsalo más adelante, no ahora: transformers, tokenizers, datasets y modelos preentrenados. [Abrir recurso](#)

Sobre YouTube: Sí, úsalo. Pero usa playlists/canales concretos para una pregunta concreta. YouTube es excelente para intuición y muy malo como roadmap si vas saltando por recomendaciones.

13. Checklist de dominio por bloques

B1 - Fundamentos

- [] Sé distinguir lista, array, vector, matriz y tensor.
- [] Puedo predecir shape y ejes.
- [] Entiendo producto escalar y cosine similarity.
- [] Entiendo media, varianza y desviación.
- [] Puedo explicar derivada, gradiente y gradient descent de forma intuitiva.

B2 - ML

- [] Distingo regression/classification/clustering.
- [] Sé separar train/validation/test y explicar data leakage.
- [] Puedo justificar una métrica.
- [] Entiendo overfitting y cross-validation.
- [] Puedo entrenar un modelo pequeño y analizar errores.

B3 - Deep Learning

- [] Entiendo tensor, batch, layer, weight, bias y activation.
- [] Sé señalar forward/loss/backward/optimizer.
- [] Entiendo qué hace autograd.
- [] Puedo escribir/leer un training loop.
- [] Puedo diagnosticar train vs validation loss.

B4 - LLM

- [] Entiendo tokenización y token IDs.

- [] Sé qué son logits y softmax.
- [] Entiendo sampling y temperature.
- [] Entiendo embedding y attention a nivel conceptual.
- [] Puedo explicar constrained decoding.

B5 - RAG

- [] Entiendo corpus/chunk/query/index/top-k.
- [] Puedo explicar TF-IDF y BM25 intuitivamente.
- [] Entiendo embeddings y cosine similarity.
- [] Sé qué mide Recall@k.
- [] Puedo separar fallo de retrieval y fallo de generation.

B6 - Agents

- [] Entiendo tool calling y schemas.
- [] Puedo dibujar el loop del agente.
- [] Sé definir estado y condición de parada.
- [] Entiendo por qué sandbox/timeouts importan.
- [] Puedo explicar MCP a nivel conceptual.

14. Qué haces mañana

No empieces por IA generativa: Empieza por B1.1. Tu primera meta no es “hacer un modelo”; es aprender a representar y manipular datos numéricos con NumPy y entender qué significa la forma de esos datos.

Sesión 1 - B1.1/B1.2 (60-90 min)

1. Abre la guía “NumPy: Absolute Basics for Beginners”. Lee solo la introducción y creación de arrays.
2. Crea manualmente varios arrays 1D y 2D en un archivo de práctica.
3. Antes de imprimir ndim/shape/size, predice cada resultado.
4. Prueba un caso que te rompa la intuición y anota la pregunta.
5. Ven al chat y dime: “Ruta IA - B1.1/B1.2. Esto es lo que creo que es un ndarray, shape y ndim: ...”. Yo te evalúo y seguimos.

Sesión 2 - B1.3/B1.4

- Indexing/slicing.
- Filas y columnas.
- axis=0 vs axis=1.
- Ejercicios de predicción de shape.

Sesión 3 - B1.5/B1.6

- Suma de vectores y escalares.
- Producto escalar a mano.
- Producto escalar con NumPy.
- Explicar por qué dimensiones incompatibles fallan.

Si una semana 42 aprieta: La ruta se congela donde esté. No compenses haciendo 6 horas el domingo. El aprendizaje de IA debe aumentar tu nivel, no competir con el Common Core.

15. Principios para los próximos 12 meses

- Fundamentos antes que frameworks.
- Una fuente principal por tema; no diez pestañas abiertas.
- Cada concepto termina en una acción.
- Los proyectos de 42 son el laboratorio principal.
- El estudio paralelo debe prepararte 1-2 pasos por delante, no diez proyectos por delante.
- No midas progreso por horas de vídeo, sino por lo que puedes explicar y construir.
- Cuando no entiendas algo, no escondas el hueco usando una librería: aíslalo y trabájalo.
- Tu objetivo no es memorizar IA. Es desarrollar la capacidad de aprender sistemas nuevos con rapidez y criterio.

Siguiente punto: B1.1 - Qué problema resuelve NumPy y qué es realmente un ndarray.